

ing for YouTube.

1. Click the [More] button and click Live on YouTube

1. Select the GitLab Unfiltered or GitLab

1. Grant permissions to the YouTube account by clicking the [Allow] button

1. Follow the instructions for starting a recorded video.

1. Click the down arrow next to "Live on YouTube" in the top left once the streaming has begun. Then click "Copy Streaming Link" and paste in the agenda for easy reference post meeting.

Video walkthough of how to Livestream to YouTube

The following 5-minute video shows how to start a livestream from a Zoom Meeting to YouTube Unfiltered.

```
<figure class="videocontainer">
  <iframe src="https://www.youtube.com/embed/mWWWdbS2q9U" frameborder="0"
allowfullscreen="true"> </iframe>
</figure>
```

If you receive a permission error when attempting to start a livestream (i.e. "Please grant necessary privilege for live streaming"), you may need to first select your user@gitlab.com account in step 4 above. It will prompt you to grant permissions, and you should do so. After that, return to step 3 but this time select GitLab / GitLab Unfiltered at step 4. If the problem persists, reach out in it-help.

Livestream with OBS

1. Install and launch OBS Studio

1. Allow OBS to automatically configure your settings:

1. Accept the recommended video settings:

1. Select "Custom" service and configure OBS to livestream to GitLab's Unfiltered channel:

1. Add your webcam as a "Video Capture Device" from the Sources tool:

Post everything

We post everything that doesn't contain confidential information to YouTube.
We found that it helps with:

- 1. Hiring because candidates can get a feel for the company by seeing meetings.
- 1. Retention because sharing reinforces our value of transparency.
- 1. Community because people feel more part of what is happening at the company.
- 1. Sales because people see our training materials they sell themselves.
- 1. Enablement because people can find detailed content about certain subjects.
- 1. Awareness because lots of time is spent on YouTube and our videos get lots of views in aggregate.
- 1. Training because content on YouTube is easier to consume even for team members, see why not Google Drive

No quality bar

You don't need to be worried that something is not of the high enough quality because:

- 1. The GitLab Unfiltered YouTube account is our casual space for communicating and sharing with the public. Content on the Unfiltered channel includes, but is not limited to, team meetings, discussions, monthly release kick-offs, office hours, CEO 101 meetings, and informal demos.
- 1. Algorithms will ensure that a video will be distributed to the right-sized audience.
- 1. We can embed videos if and where they are relevant, for example in docs.
- 1. Enhance videos later if we need to (trim it so it starts immediately, add transcript, extensive description, links to relevant materials).
- 1. Follow up later if we need to (rerecord, do an interview).

No extra work

Creating content for the Unfiltered channel shouldn't be extra work.

You should do what you normally do.

But when a meeting is possibly interesting for more people, make it a livestream for the Unfiltered channel.

Or when you start talking about something possibly relevant to more people, hit the record button.

Make private quickly

In case there are any concerns raised about a video on the Unfiltered channel, everyone has the authority to make it private.

You don't have to wait for any permission, just go to the channel specific YouTube Studio, here is the one for unfiltered and mark the video private.

Please note that there is another tab for live videos.

If you mark a video private when it doesn't need to be, it is be easy to undo: simply mark the video public.

Don't share secrets

Secrets (e.g. access tokens, private keys, passwords, and credentials to systems running on GitLab infrastructure) and GitLab team member personal data (phone numbers, home address) should never be shared in public. Any video that exposes secrets or team member personal data should not be shared as a public YouTube recording.

Similarly, internal-only documents and private or confidential issues should not be shared publicly in YouTube videos.

If a video contains secrets or sensitive information:

1. Do not publish the video on YouTube or any other public platform.
2. Immediately revoke any secrets that were exposed in the video.
3. For other sensitive information like personal data, redact the information using the Blur feature in YouTube Studio.
4. Only after all secrets have been revoked and sensitive information redacted, you may proceed with publishing the video.

Remember that blurring is not a substitute for revoking exposed secrets. Always prioritize revoking secrets over relying solely on blurring.

If a video containing secrets or non-public information is accidentally uploaded to the Unfiltered channel, make it private quickly and use /security in Slack to engage the security engineer on call to follow up on the leak.

Videos which initially exposed secrets or internal information can be made public again only after ensuring that all secrets have been revoked and sensitive information redacted.

Why not Google Drive

Google Drive is not intended to process video distribution and, as a result, lengthy uploading and processing times are common. Google drive can create a number of issues including the following:

1. Long upload times (in some cases up to 10 hours for longer content) that risk failing before completion.
1. Extended processing time, as Google Drive is not intended to quickly turn around video for playback.
1. Compromised playback quality.
1. Not inclusive (for example, no caption support)
1. Not indexed for searching.
1. Less transparent to both other employees and to the public.
1. Difficult to view across multiple devices.
1. No support for other critical features such as playlist, chaptering, descriptions, embedding, or time codes.

Always use YouTube and never use Google Drive, even for private videos, because YouTube videos:

1. are streamed more reliably.
1. have mouse-over thumbnails.
1. can be played at a higher speed.
1. can be fast forwarded and rewound in 10-second blocks.
1. can be time-shifted by adding them to a watch later list.
1. can be embedded, for example in the handbook.
1. restart at the right spot after being reloaded.
1. can be easily viewed on other devices, like TVs or streaming devices, with YouTube support.
1. allow links to a specific time in the video.
1. can have subtitles added automatically.
1. are zero rated by some mobile providers
1. will be served to people when it is relevant, automatically, since YouTube is a distribution channel.
1. allows anyone to contribute by leaving comments.
1. public videos show up in Google search.
1. easy to make public if the video is suitable for that.
1. have playlists that allow for better organization of the video content.
1. show the number of views, so you can see what resonates
1. have the ability to comment, so people can contribute
1. have voting so you can review what videos aren't meeting expectations

Visibility

There are three types of visibility:

1. Main
2. Public
3. Private

The level of visibility should be displayed clearly in the titles of calendar invites.

For example, when you do a livestream to the public channel say: "Public stream" at the start of the invite.

You can skip live in livestream since every stream is live.

Do not say Unfiltered stream since it is ambiguous.

Do not say just livestream since people don't know what the level of access is.

We don't post:

1. Unlisted videos (see the "never unlisted" section below).
2. Private videos on our branded channel, all videos on that channel are intended for a wider audience.
3. Use Google Drive for any video, since it has many drawbacks.

Never unlisted

We never post unlisted videos on any channel: it is too insecure for sensitive materials and not findable enough for public materials.

Sometimes, some videos are still posted as unlisted, they can be spotted with this link (while

using the GitLab Unfiltered account).

You can change the visibility of videos by clicking on the Details buttons when you hover a video in the list linked above.

Unable to view a video on YouTube

If you get a "Video unavailable This video is private." error on youtube, the error may be caused because the video is private to GitLab Unfiltered and you are accessing the video with your individual YouTube account.

In order to see the video,

1. Click your photo or icon in the upper right corner of YouTube.

1. Then Click > Switch Account

1. Now Select > GitLab Unfiltered. Do not select the account with your name. If GitLab Unfiltered is not an option, see Access

1. Once you are on GitLab unfiltered, reload the page and you can view the video

Check out this video for a visual walk through of how to access private videos on GitLab Unfiltered.

Access

1. GitLab branded channel: To request access, create a Single Person Access Request issue and assign the listed provisioner (column G) from the Tech Stack YAML.

1. GitLab Unfiltered: Everyone should get access to YouTube Unfiltered during onboarding. If you do not see an invitation in your Inbox, please check the Pending Invitations section of your Google Workspace account. If your invitation is not there, please request a new invitation with the People Connect Team via HelpLab.

For People Connect troubleshooting:

1. If a team member reaches out that they have not received their GitLab Unfiltered invite, resend the invite via the Manage Permissions

1. Should the team member still not see the invite under Pending Invitations, revoke the original invite by doing command f in the Manage Permissions section for the team members email address. You can revoke by clicking the X next to the email address.

1. Once you have revoked the invite, close the YouTube Unfiltered page and reopen.

1. Once reopen, select Manage Permissions and reinvite the team member by clicking on the Add option and enter the team members email address.

1. This should now appear in the Pending Invitations section here.

Commenting

Please remember that if you find a user comment to respond to on one of our channel videos, that it will be perceived as official communication from the company. In almost all cases, unless your job specifically requires you to interact in a community manager role or capacity, it is best to switch to a personal account prior to engaging with a video or user on YouTube.

Consider switching to your personal account, responding to the comment, and identifying yourself as a GitLab employee offering help.

Please do not comment on videos from non-GitLab YouTube accounts using any company related YouTube profile. Commenting as GitLab is reserved for responses to our owned videos on our channels only.

Subscribing to Channels

The GitLab Unfiltered YouTube account should not be subscribed to any channels except:

- @Gitlab.

House cleaning

The GitLab Unfiltered YouTube account is a shared space. Sometimes team members might accidentally comment or subscribe to channels thinking they are on a personal account.

If you see a channel subscription or comment that violates the rules of this handbook, please take a bias-for-action to keep the GitLab Unfiltered YouTube account pristine. Delete any accidental channel subscription or comment to return the YouTube account to a pristine state. If you are unable to remove the relevant comment or subscription, please notify the People Connect Team immediately via HelpLab.

Organizing

Our YouTube channels contain a lot of content.

As a result, it can often be hard to find videos once they are published, particularly on the Unfiltered channel.

Make liberal use of tags and playlists to organize the video content you publish.

For published recordings of recurring meetings, which should be posted on the Unfiltered channel, please search for a playlist to put the video in. If there is no applicable playlist, please create one. You can also create a playlist that would automatically add new videos containing certain text in their titles, descriptions or tags to avoid manual steps (instructions)

Audience on YouTube

When publishing, YouTube will ask if the content is made for kids. GitLab content should be marked as "No, it's not made for kids".

Never call it a livestream or just a recording

It is essential that we always know if something is public or private.

Always mention that in the calendar invite title.

Therefore say:

- Public stream
- Private stream
- Public recording
- Private recording

Never say: livestream or recording since those are ambiguous. See top misused terms.

What if your aren't sure whether the stream is Public or Private?

There are some recordings where it is not clear whether they should be Private or Public or they require a review by the Legal team before posting publicly on YouTube. It would be disingenuous to tell our team that the recording is Private and then change the recording to Public. In this scenario, we should state that this a Reviewed Stream or a Reviewed Recording. This stream will be initially Private and upon Legal review could be changed to Public.

Starting a Livestream

The below instructions apply to livestreams to YouTube.

1. The moderator asks host, 'I am getting ready to start the public/private stream. Are you ready to begin speaking as soon as you see the YouTube Icon in the top left corner of Zoom?'. Please make sure you mention public or private.
1. After the hosts confirms the Moderator clicks Live to YouTube and start the livestream.
1. The host should do an introduction stating who they are, what their role is, and what the meeting is about, e.g. "I am Jane Doe, the VP of Widget Production, and today's Group Conversation is about the molding phase of widget production."
1. Please note that if the recording starts a 'this meeting is being recorded' message the host should talk over that because otherwise the recorded video will needed editing to prevent an awkward pause. Livestreaming doesn't trigger this message, only recording does.

Useful links

- My Live Events: schedule and review upcoming live events
- YouTube live streaming introduction page - Here you can check if your channel meets all requirements for live streaming
- YouTube tutorial on setting up a live stream - If you need extra help or information on setting up a YouTube live stream in general
- YouTube live dashboard - Here you can find your encoder setup variables, manage your stream and see your chat

Uploading conversations to YouTube

```
<iframe width="560" height="315" src="https://www.youtube.com/embed/xGwX9zjNr2E?rel=0"
frameborder="0" allow="autoplay; encrypted-media" allowfullscreen></iframe>
```

1. Default to public or private streaming instead of uploading.

1. Please follow the process for starting the recorded video.
1. Unless stated or arranged otherwise, our expected behavior is that the meeting organizer is responsible for distributing the video after the meeting.
1. Log in to the Zoom account of the meeting and go to the menu on the right and choose "My Recordings" (it can take up to 30 minutes before the recording is available to be shared).
1. Select the meeting and download the recording to your computer (if you can't find the recording because it was a while ago check "Trash" in the menu on the top left and "Recover" the recording).
1. Go to the YouTube upload page and log into the channel. If you're logged into your personal YouTube account, you may need to first log into your GitLab YouTube account to access the channel. If you're already logged into your GitLab YouTube account and have access to the channel, you need to switch accounts to that channel. To switch accounts, click on the account icon in the top right corner and then click "Switch account".
1. Drag and drop your recording into the window to upload it. Keep the privacy dropdown on the default 'Public' setting (unless there is confidential material). Use the dropdown menu to set the video to private if it the video contains confidential material.
1. While it's uploading, edit the title and description. Place "Confidential:" at the beginning of the video's title if the video will be kept private on our YouTube channel.
1. Be sure to include relevant links (for example a handbook page or presentation) in the description, and add the video to any relevant playlists.
1. In the "Comments and ratings" section, select "Hold potentially inappropriate comments for review" - this allows all valid comments to be posted, allowing everyone to contribute, but holds spam or derogatory comments for review. There may be videos where we do not wish to open the comments section or need to moderate each comment. If you think your video would potentially need to go this route, add your question to the marketing Slack channel.
1. When it is done uploading, press publish, then click on the Embed tab and copy the code, and insert that in the relevant part of the handbook or documentation.
1. After uploading a video to the GitLab Unfiltered channel, be sure to add a link to the new video in the content-updates channel in Slack.

Don't worry about the quality

1. There is no minimum quality, so please share it on our GitLab Unfiltered YouTube channel, as long as there is nothing inappropriate or confidential.
1. Everyone at the company probably has at least one conversation every week that is relevant to more people, so please share it.
1. We always list videos publicly instead of having them unlisted, unless there is confidential material. This allows more people to find the content. If the material is confidential, set the video to private.
1. Don't worry about whether or not it will be interesting to absolutely everyone. Just give it a descriptive title so people know what it is about, and let them decide whether or not they should watch it.
1. Make sure that all participants are aware that you're recording.
1. You don't have to be sure it is interesting and OK to share when you start recording; you can make that decision after the fact.
1. If you record an in-person conversation with your mobile phone please hold your phone in landscape (horizontal) mode.

Advanced setups

Depending on your needs, you might want use software to provide overlays and/or reroute audio. 99% of the people at GitLab don't use this, but see below for instructions if you do want to use it.

Encoder

- OBS Studio Win/Mac/Lin Open Source or install with 'brew install --cask obs' on mac

Audio Rerouting

- Soundflower Mac Open Source or install with brew install --cask soundflower
- Loopback Mac Closed Source - Great closed source alternative
- Windows alternatives
- Linux alternatives

Setup

- Install both OBS Studio (encoder) and an audio rerouting software.
 - Check if your channel is ready to live stream.
 - Copy your encoder variables en setup your live streaming environment with the YouTube live dashboard
 - Open up OBS Studio and open preferences
 - Input your encoder variables under "Stream"
-
- Configure your to-be-recorded software's audio to be rerouted to an alternative audio source/output.
 - Under "Audio" in OBS Studio preferences, select the alternative audio source/output under Mic/Auxiliary Device 2. See this page for more help
 - Under "Video" select your preferred resolution and fps settings. Please take note that these should optimally reflect video aspect ratio such as 1920x1080.
 - Optionally you can set up some output options under "Output"
 - In the normal window of OBS Studio you should now see 2 audio sliders, including "Mic/Aux 2". Configure these to your liking.
 - Select "Start Streaming": You are now streaming
 - Go to your YouTube live dashboard to see your live stream and interact with your viewers.

When to record and publish to YouTube

```
<iframe width="560" height="315" src="https://www.youtube.com/embed/RB8OC70RAfo?rel=0"
frameborder="0" allow="autoplay; encrypted-media" allowfullscreen></iframe>
```

SECTION: marketing/project-management-guidelines/_index.md

```
{{< include "includes/wip-notice.md" >}}
```

Sub Pages

1. Organization - Groups and Projects
2. Labels
3. Epics
4. Milestones
5. Managing Commitment
6. Issues
7. Boards

<i class="fas fa-toolbox fa-fw color-orange font-awesome"></i> Marketing Project Management Guidelines

Marketing uses GitLab for agile project management including groups, projects, epics, roadmaps, issues, labels, and boards. Read through the documentation on each of these GitLab features if you are unfamiliar.

Integrated Campaigns

Marketing Departments collaborate to produce Integrated Campaigns. An Integrated Campaign is a communication effort that includes several campaign tactics such as blog posts, emails, events, advertisements, content on about.gitlab.com, videos, case studies, whitepapers, surveys, social outreach, and webcasts. An Integrated Campaign will have a campaign theme that summarizes the message we are communicating to our market.

Active integrated campaigns

Active integrated campaigns

Have a new campaign idea? Make a suggestion

Project Management Processes

We cultivate a deep understanding of our own product by using GitLab to manage our planning, collaboration, and execution of Marketing activities.

The latest Project Management recommendations can be found here from FY21-Q2 Agility Project

Milestones

The latest recommendations for Milestones from FY21-Q2 Agility Project

Weekly Sprints

Within the www.gitlab-com repo (parent repo to Marketing) there are weekly milestones, which some teams use plan a weekly sprint cadence. Each of these sprints begins with "Fri:" for the Friday upon which that sprint ends, making them searchable in a list here.

Each week on Monday, any open MRs and issues still assigned to the previous week's milestone are bulk moved forward to the next week, and the previous milestone is closed out. This is a manual process currently performed by Danielle.

Groups and projects

The latest recommendations for Groups and Projects from FY21-Q2 Agility Project

1. The Marketing Group houses all marketing projects.

1. Labels should be created at the group level so they can be used in all projects within Marketing group.

- Labels should not be duplicated in individual projects. It creates board/tracking conflicts.

1. The following are the approved marketing projects, CMO approval is needed to begin a new project.

- Product Marketing (includes PMM, TMM, and Market Insights)
- Developer Relations
- Digital Marketing Programs
- Demand Generation
- Corporate Marketing (includes content, website, corp events, brand, etc. - old location in deprecated group, to be updated)

- Marketing Operations

- Field Marketing

- Account Based Strategy

- Sales Development Organization

- Digital Advertising - used for external advertising agency

- Onboarding

- Contribute - this is a sub-group that contains all projects for each Contribute event

- Growth Marketing

- Partner Marketing (includes both tech partner and channel marketing)

1. Issues should be logged in the team project ultimately responsible for completing requested work. (i.e. If SDR needs list uploaded -> issue created in Marketing Operations project).

Issues, Milestones, and Epics

The latest recommendations for Epics and the latest for Issues from FY21-Q2 Agility Project

1. Each issue represents a discrete unit of work with a deliverable. For example 1 2 3

1. Every MR should have an issue so that it can be tracked on issue boards.

1. Milestones represent units of work to be completed within a specific time frame, sometimes referred to as sprints. They are comprised of multiple issues that share a common due date, and help break large projects into more manageable parts.

1. Epics represent projects that comprise multiple issues. (Don't use "meta" issues for this purpose. If you have an existing meta issue you can promote them to epics using the /promote quick command.)

- Epics live at the group level (e.g. issue from multiple marketing projects can be added to an epic.)

- Epics are labeled with a group label of the team that owns the epic.

1. The top 3-5 strategic initiatives are tracked in epics using the CMO label. (Don't apply the CMO label to other epics.)

1. Roadmaps are used for time-based display of epics with a start and end date. (for example, events and time-based campaigns.)

Boards and Labels

The latest recommendations for Labels and the latest for boards from FY21-Q2 Agility Project

1. Each team has one or more boards to track ongoing workstreams.

1. Generally, create a board for each function. (For example PMM has boards for Sales Enablement, Analyst Relations, Customer Relations, etc.)

1. Each board uses a standard set of columns/labels so that folks can easily understand what is happening on another teams board.

1. The board labels use group scoped labels with mktg-status:: and one of four statuses. Status labels should be used on all issues within the Marketing group:

- mktg-status::plan - work that is proposed, in an exploratory state.

- To exit the plan stage the work must be assigned to a DRI.

- DRI accepts responsibility for the task by changing the label from mktg-status::plan to mktg-status::wip and creating a merge request (MR), if appropriate. The plan status is optional, as issues that don't require formal planning can be opened and labeled mktg-status::wip.

- mktg-status::wip - work in progress that has been accepted and assigned to a DRI.

- Work in this stage should not be merged.

- Merge requests (MRs) should be prepended with WIP:. At GitLab we allow reviewers to start reviewing right away before work is complete.

- Use MVCs: At any time, if the work is complete enough that merging would be better than what current exist the issue should be labeled with mktg-status::review and WIP: should be removed from the title.

- Optional: mktg-status::review - work has been completed enough that it is ready for formal review and approval.

- Work that is approved can be either merged or scheduled.

- The review status is optional.

- Work that doesn't require review can simply be merged/closed.

- Optional: mktg-status::scheduled - work that is complete but should be scheduled for a future date.

- Scheduled status is optional as not all work will need to be scheduled.

- closed - when work is delivered the issue should be closed.

1. Don't duplicate status labels at the project level. Use group labels (at the Marketing Group level) as much as possible.

Department Labels

Each Department within Marketing can have "additive" labels - meaning they are used to enhance the tracking and workflows for that respective team. These "additive" labels are used in conjunction with the broader Marketing labels. The Department label usage is documented on each of the respective handbook pages:

- Marketing Programs

- Operations

- Sales Development Organization

- Field Marketing

- Account Based Marketing

Default Issue Text

All of the projects within the Marketing subgroup include default issue text to ensure the

Department labels are applied consistently and broader adoption of the global Marketing labels.

Using Default Issue Text

When a new issue is opened in any project, the issue description will contain a small snippet of text applying that teams' label & Marketing scoped mktg-status::plan label.

- If a template is chosen, a message will appear confirming you want to change the text w/in the issue description, click Apply Template and continue as normal

Updating Default Issue Text

The default text is minimal and generic. Any team can make the collective decision to update the text. Access to modify the text may be limited based on group/project permission level, if you do not have access to the General settings section, please reach out to @mktg-ops via slack.

Please note this is not an issue but there is no dynamic functionality. The default issue text applies to all issues opened within that project and the text should be broad enough to encompass a roles within that team.

1. Navigate to project to be updated

1. Left side menu, hover over the wheel widget (last icon) -> Select General

1. Scroll down & select Expand next to Default Issue Template

1. In the text box you can add any markdown formatting & modify the text. The text that has been added included several lines above it, so it may appear to be a blank box. Scroll down &/or expand that text box to see complete text.

- Please do not delete the labels section.

- Label section can be updated to include more labels &/or switch the Department label

- Important to leave the mktg-status::plan label in that section

1. Once edits have been made, click Save changes. The changes be applied immediately to any new issue opened.

- Does not affect already opened issues.

If you have any issues &/or questions, please reach out to the MktgOps team (@mktg-ops) via slack.

How it all fits together

Figuring out how where and how to create a board, epic, label can be confusing. The following diagram is a very high level example of how it all fits together. If there are questions please ask in the mktgops slack channel (must be GitLab team member for active link).

SECTION: marketing/project-management-guidelines/boards.md

{{< include "includes/wip-notice.md" >}}

Background

Boards in GitLab make it possible to visualize and manage lists of issues which can be defined by one of three ways: Labels, Team members, and Milestones.

Key things to know

Board usage

1. First, a common use of boards is in the Kanban style of visualizing work

- In this scenario there are three or four lists: Backlog; Doing; Waiting for; Done
- A user would move an issue from the Backlog to the Doing list to indicate they are working on the specific issue. If they finish the task, then they would move the item to Done, and if they ran into a barrier and had to wait for something outside their control, then they would move it to Waiting for
- In this simple example, the Board is essentially adding and removing labels when the issue is moved from one list to another.

1. A second situation where boards are useful is assigning work to an individual. In this situation the board is defined by team members, where different users are represented by the lists. When an issue is moved to a user's list the issue is then assigned to that user.

1. The third example of how boards can be used is where the lists are defined as different milestones, and moving an issue from one list to another, effectively changes which milestone when the issue will be worked on.

Board Details

The scope of issues that are displayed on a board are filtered either by label, assignee, milestone, or possibly weight.

Boards support groups of lists that are defined by a combination of

1. A single label
2. A single milestone
3. An individual

When an issue is moved from one list to another on the board, the board basically does two things:

1. It removes the label, milestone or assignment from the starting list
2. It adds the label, milestone, or assignment of the list the issue is added to.

```
{{% alert color="info" %}}  
<i class="fas fa-info-circle" aria-hidden="true" style="color: rgb(49, 112, 143)  
;"></i> Board lists also sum up the total "weight" of the issues in each list, as a way to  
estimate the total work in the list.  
{{% /alert %}}
```

Board hierarchy

Boards can exist at BOTH project and group levels in GitLab.

Depending on the scope of the issues which need to be managed, the board should be defined at the most appropriate level.

Issue priority

The order of issues on Board Lists can be arranged by dragging and dropping them and be used to determine the relative priority of the related issues. When an issue is moved vertically on a board, this changes the relative relationship of the issue on ALL boards in GitLab.

Stay tuned for future features

We're investing in boards to add cool features like

- swimlanes
- advanced editing of Issues and MORE

Known limitations

1. Moving issues from one list to another on a board ONLY makes one change (adding/removing a label, assigning/unassigning an issue).
1. It is not possible to drag an issue from one list to another and have MORE than one thing change. For example, you cannot Assign an issue to a person and ALSO add a label or milestone using a board.
1. You cannot sort issues in board lists. Only drag up and down.
1. Whenever ANYONE changes the order of an issue on a board, there is NO history or record of the change in priority. So, if a team is managing issues by dragging up and down to indicate importance, then there is currently no record of who changed what and when it changed. The relative priority/importance of related issues is not exposed in the GitLab UI.
1. There is no way to have a 'read only' or 'static' board to view related issues. It is always live and ready to accept changes.

Guidelines

Define Boards at the lowest level needed

If the relevant issues that need to be managed in a board are at the project level, then create the board at the project.

Define Workflow and Iteration boards for each Marketing group

1. There should be a minimum of 2 boards per Marketing group: Workflow and Iteration.
1. Each group should have its own label to be able to create the board
1. Each issue the group is working on should use at the very least that label
1. These two main issue boards should be listed on each team's section on the Marketing handbook landing page.

Workflow board

Goal: describes the task status for a given sprint.

- Generally filtered by team label
- Board scope set to the current sprint
- Can be further filtered by assignee.
- Uses the mktg-status:: scoped labels as lists
- Workflow board example

Iteration board

Goal: describes the planned workload through iterations

- Can be further filtered by assignee
- Uses the main marketing iterations as lists
- Backlog is a milestone (permanently dragged to the left, and collapsed)
- Optionally: at their discretion groups can add a milestones board for project milestones
- Iteration board example

SECTION: marketing/project-management-guidelines/commitment.md

```
{{< include "includes/wip-notice.md" >}}
```

Background

Teams and individuals are often asked / requested to work on multiple efforts, across the company. For example,

- An event, needs booth branding or messaging
- A campaign, needs positioning/messaging and perhaps a gated white-paper
- A new whitepaper or infographic needs design input
- A campaign, needs a customer case study
- A team wants an analyst inquiry

While it is possible to use issue assignments or labels to track simple tasks and assignments, in an organization of scale, there are challenges.

1. Any individual can be assigned to an issue. There no a built in mechanism to manage or moderate assignment of a person to work on an issue.

2. If labels are used to indicate a team is responsible for contributing to an issue, there is no current mechanism or process to manage adding a team label to an issue.

These two limitations make planning and managing a team's commitments to tasks and work a challenge.

Solution - Commitment management

Use a project issue and workflow to consistently capture requests and manage commitments. Effectively a simple and efficient request-issue that enables a request to be made, and a process to answer the request.

Commitment Management overview

1. Simple request issue (defined by an issue template)

1. A simple set of predefined workflow steps (using GitLab scoped :: labels)

1. The team/leadership that receives the request manages the process and takes action

The typical actions/responses to a request

- a. Backlog - the request is acknowledged, but not actively worked right now
- b. Assigned - the request is accepted and someone from the team is DRI for the work
- c. Transferred - the request has been shifted to a different team
- d. Declined - the request is not accepted
- e. Canceled - the request was canceled
- f. Completed - the request is done and delivered

The typical workflow for a request

mermaid

graph TD

```
A["New Request &128229;"]::Open --> B["Backlog &129300;"]::Open
B --> C["Assigned &129418;"]::Open
C --> D["Completed &128076;"]::Close
A --> E["Transferred &128666;"]::Close
A --> F["Declined &128721;"]::Close
C --> G["Canceled &128533;"]::Close
B --> G
```

classDef Open fill:D8BFD8, stroke: 333

classDef Close fill:c4d3d9, stroke: 333

mermaid

graph LR

subgraph legend["Legend"]

GG(["Open Issue"])::Open -. PP["Issue Closed"]::Close

end

classDef Open fill:D8BFD8, stroke: 333

classDef Close fill:c4d3d9, stroke: 333

style legend fill:FBFBF2, stroke: b0b0a9

Labels define the process

There are two key labels that allow a team to define and manage the process:

1. A project label that tags all issues in the process for example smrequest - is the Product and Solution Marketing label for all requests. Need to explore the potential of these being common group level labels (shared across projects).
2. A set of scoped labels that define the workflow. Here is the list of labels from the Product and Solution Marketing workflow:

- smreq::New
- smreq::Backlog
- smreq::Assigned
- smreq::Completed
- smreq::Declined
- smreq::Transferred
- smreq::Canceled

Typically - you would replace the "sm" with the team initials.

Important Note: Managing a process like this based on labels, can create overhead in keeping everything in the right status. Expecting everyone to memorize all labels and right steps might be a challenge. It is possible to automate rules/policies that will either automatically close issues, add a label, etc. Learn how to automate label hygiene

An Issue Template Starts the process

Using an issue template helps to consistently define the process and make it easy to get started.

- Issue Templates can also be directly linked to via a URL which makes it easy for a person requesting help to quickly open an issue template.

For example this URL will open a Product and Solution Marketing Request:

- <https://gitlab.com/gitlab-com/marketing/product-marketing/issues/new?issuabletemplate=A-SM-Support-Request>

The key is the end of the URL new?issuabletemplate=A-SM-Support-Request which tells GitLab to open a specific template!

The issue template, both describes the information that is needed to understand and prioritize the work, but also includes Quick Actions to assign specific labels. At the bottom of the strategic marketing template there are the following quick actions to add labels:

```
/label ~"smrequest" ~"Product and Solution Marketing" ~"smreq::newrequest" ~"mktg-status::plan"
```

How Commitment Management Works

1. A person leading a project, campaign, event, or whatever needs help from a team.
1. They open a request issue and provide the background
1. A DAILY Process - The team (leaders or the team in general) triages the New Requests and decides how to respond. (backlog,assigned, transferred, declined)
1. When the request is actually assigned to a team member, the label req::assigned is applied and the individual is assigned to the request issue.
1. The assigned individual does the work. (Either in the request issue, or in the project where the request originated - depending on what makes the most sense.)
1. When the work is done, the issue is labeled req::completed (the SM Triage Bot automatically closes completed issues)

mermaid

sequenceDiagram

Requestor->>+Team: Can you support this

participant Requestor

Note right of Requestor: ::newRequest

Team->>Team: What is the priority of this request?

participant Team

Note right of Team: ::Backlog

Team->>-DRI: Here is an item to work on!

participant Team

Note right of Team: ::Assigned

DRI-->>Requestor: I'm working on it

DRI->>Requestor: I'm Done

participant DRI

Note right of DRI: ::Completed

Guidelines

To manage commitment, use a commitment tracking issue

Just adding a label or tagging another person is NOT the same as a commitment!

1. Create a commitment tracking process (scoped labels, issue template, and team processes)
2. Apply the process to capture and manage the team's work. As a principle - if there isn't an accepted and assigned request issue, then there isn't a commitment.
3. Manage requests, build and curate a backlog, assign and deliver work.

SECTION: marketing/project-management-guidelines/epics.md

{{< include "includes/wip-notice.md" >}}

Background

Epics provide a way to organize and manage a set of issues and sub-epics that share a strategic

theme. In addition to logical grouping, epics enable project managers to perform higher level planning and build a roadmap with visual status tracking.

Key things to know

- 1. Epics are defined at the group level.
- 1. Epics can be made confidential.
- 1. Epics can contain both issues and epics as children.
- 1. Epics can be used as a filter in issue lists and issue boards.
- 1. Epics provide visibility on child epics, issue statuses, and the roadmap timeline (Gantt chart).
- 1. An epic is visible on a roadmap view if it contains a start or due date - all marketing roadmap here.
- 1. Child epics are also visible on a roadmap view, nested under their parent, when they have a start or due date.
- 1. The roadmap view provides a timeline of epics and their completion status based on aggregate issue weight completion for issues linked to the epics.
- 1. The roadmap view is available on the individual epic and the group page under Epics > Roadmap. The group page view provides additional filtering, sort order, and timeline units for display.
- 1. An issue can only be the child of a single epic.
- 1. Epics can have multiple child epics up to a depth of seven levels in total.

Known Limitations

- 1. Epics cannot be created from templates (issue) however, there are three workarounds.
 - 1. First - Epics Can be created from an issue that is promoted to an Epic (in this case, an issue template could be a substitute for an epic template).
 - 1. Second - The mixture of issues related to an epic can be templated in a spreadsheet and uploaded to populate the issues related to a given epic. See this epic template overview - note for collaborative epics that span multiple GitLab projects/teams: with this process, for the total number of projects/teams the issues span, you will need to complete that number of uploads of the spreadsheet, broken out by project/team.
 - 1. Third - Include code for the epic in the relevant handbook page to be copy/pasted, with issuable template hyperlinks. See this handbook page for an example.
- 1. Epics do not have an assignee.
- 1. Epics cannot be created in projects (issue).
- 1. Epics cannot be cloned (issue).
- 1. Epics cannot contain issues from above their parent group (epic).
- 1. Feature request: calendar view (issue).

Guidelines

Use the roadmap view for high-level overview

The roadmap view can serve as the single source of truth and as a high-level overview to effectively understand the strategic themes, OKRs, marketing projects, and campaigns the team is working on.

A single link to the parent epic or to a group's roadmap page with (optional) filtering can be used for leadership visibility.

Milestones and issue boards can then be used to drill down into the details for more granularity on individual task status.

Associate all related epics in an ancestor relationship

All epics that are related to an OKR epic or a strategic theme epic should indicate just one of them as the ultimate ancestor in order to align them properly on the roadmap.

Set the start and end date for epics

In order for the roadmap view to be displayed and to provide meaningful visual timeline information, explicitly add the start and due dates to epics during the planning process.

Use executive theme labels

All epics that are part of a strategic theme or an OKR should carry the associated executive label, so that they can be filtered on the roadmap view. Optionally they can also carry one or more of the approved group-level labels.

Define weight for child issues

As an additional piece of information to track completion status, the roadmap offers a completion percentage based on the weight of the children issues that have been completed over the total issue weight of all direct descendant issues combined.

It's recommended to set the relative issue weight for all child issues to be able to track and report epic completion on the roadmap view.

Set health status for child issues

It can be good practice to highlight issues that need attention or might be at risk before a status review. This information can then be displayed on the roadmap view. For this to work, a health status can be assigned to individual issues.

You can set health status in the UI or with the `/healthstatus` and `/clearhealthstatus` quick actions.

SECTION: marketing/project-management-guidelines/groups.md

{{< include "includes/wip-notice.md" >}}

Background

GitLab helps to organize teams and work through a hierarchy of Groups and Projects.

Key things to know

Groups can contain other groups (subgroups) and projects.

Groups and Projects are both similar and fundamentally different, which can be confusing when using GitLab

Feature	Groups	Projects	Comments
Epics	X		Collection of related sub epics and issues in a strategic theme
Roadmaps	X		Graphical view of epics over time
Milestones	X	X	Burndown charts of a time boxed period
Issue Insights	X	X	Analytical view of issues and merge requests
Labels	X	X	Flexible ability to tag issues, epics and Merge Requests
Issue Lists	X	X	Lists of all issues, enables bulk updates
Issue Boards	X	X	Visual boards of issues grouped in lists
Issues		X	An item of work, a deliverable, a request, a discussion
Repositories		X	A set of files that are under version control
Merge Requests		X	The discussion/management of change to files under version control
CI Pipelines		X	Automation of build and test of files / code that is being changed

Graphically, this illustrates the difference between groups and projects:

Known limitations

1. Epics can only be created at the group level, not at the project level.

Guidelines

Marketing GitLab Structure

In Marketing, we have diverse teams, where some need to develop demos and code, and others need to manage complex projects such as campaigns and large events. In order to support a variety of activities, we are using Sub-Groups to provide flexibility enable the teams to have an area to work and be productive.

mermaid

graph LR

```
A0(["GitLab.com &128101;"]):::GRP
A0 --> A1(["Marketing &128101;"]):::GRP
A0 --> A2(["Sales &128101;"]):::GRPNMKTG
A0 --> A3(["Engineering &128101;"]):::GRPNMKTG
A0 --> A4(["... &128101;"]):::GRPNMKTG
A --> B(["Corporate Marketing &128101;"]):::GRP
A --> C(["Growth Marketing &128101;"]):::GRP
A --> D(["Revenue Marketing &128101;"]):::GRP
```

```
A --> E(["Demand Generation &128101;"]::GRP
A --> F(["Product and Solution Marketing &128101;"]::GRP
```

```
classDef GRP fill:D8BFD8, stroke: 333
classDef GRPNMKTG fill:F7ECF7, stroke: 333, stroke-dasharray: 5, 5
```

Marketing Sub Groups and Projects

- In order to work and manage issues, there must be at LEAST one project under EACH of the Marketing SubGroups.

```
mermaid
graph LR
    A(["Marketing &128101;"]::GRP
    A --> B(["Corporate Marketing &128101;"]::GRP
    A --> C(["Growth Marketing &128101;"]::GRP
    A --> D(["Revenue Marketing &128101;"]::GRP
    A --> E(["Demand Generation &128101;"]::GRP
    A --> F(["Product and Solution Marketing &128101;"]::GRP
    B --> BA("Corporate Events &9881;")::PRJ
    B --> BB("Developer Relations &9881;")::PRJ
    B --> BC("Corporate Communications &9881;")::PRJ
    B --> BD("Developer Relations &9881;")::PRJ
    B --> BE("All-Remote &9881;")::PRJ
    C --> CA("Global Content &9881;")::PRJ
    C --> CB("Inbound Marketing &9881;")::PRJ
    C --> CC("Brand & Digital Design &9881;")::PRJ
    D --> DA("Sales Development &9881;")::PRJ
    D --> DB("Field Marketing &9881;")::PRJ
    D --> DC("Account Based Marketing &9881;")::PRJ
    D --> DD("Marketing Operations &9881;")::PRJ
    E --> EA("Marketing Programs &9881;")::PRJ
    E --> EB("Digital Marketing &9881;")::PRJ
    E --> EC("Partner & Channel Marketing &9881;")::PRJ
    F --> FA("Product Marketing &9881;")::PRJ
    F --> FB("Technical Marketing &9881;")::PRJ
    F --> FC("Market Research and Customer Insights &9881;")::PRJ
    F --> FD("Competitive Intelligence &9881;")::PRJ
```

```
classDef GRP fill:D8BFD8, stroke: 333
classDef PRJ fill:c4d3d9, stroke: 333
```

```
mermaid
graph LR
    subgraph legend["Legend"]
        GG(["Group &128101;"]::GRP -. PP("Project &9881;")::PRJ
    end
classDef GRP fill:D8BFD8, stroke: 333
```

```
classDef PRJ fill:c4d3d9, stroke: 333
style legend fill:FBFBF2, stroke: b0b0a9
```

- As needed, the marketing teams can create additional SubGroups and projects to organize and manage their work.
- For example of a Marketing SubGroups such as under the Corporate Marketing group
 - Contribute group and
 - Contribute-Workshops group
 - Tech. Evangelism group
 - both subgroups have projects and support management of corporate events.

```
mermaid
graph LR
  A(["Marketing &128101;"])::GRP
  A --> B(["Corporate Marketing &128101;"])::GRP
  B --> BA(["Corporate Marketing &9881;"])::PRJ
  B --> BB(["Contribute &128101;"])::GRP
  B --> BC(["Contribute Workshops &128101;"])::GRP
  B --> BD(["Tech Evangelism &128101;"])::GRP
```

```
classDef GRP fill:D8BFD8, stroke: 333
classDef PRJ fill:c4d3d9, stroke: 333
```

```
mermaid
graph LR
  subgraph legend["Legend"]
    GG(["Group &128101;"])::GRP -. PP(["Project &9881;"])::PRJ
  end
classDef GRP fill:D8BFD8, stroke: 333
classDef PRJ fill:c4d3d9, stroke: 333
style legend fill:FBFBF2, stroke: b0b0a9
```

```
---
## SECTION: marketing/project-management-guidelines/issues.md
```

```
{{< include "includes/wip-notice.md" >}}
```

Background

Issues are a core building block in GitLab that enable collaboration, discussions, planning and tracking of work.

Issues are typically used to

- Support a Discussion on a specific topic
- Define requirements for a new feature
- Organize work on a specific deliverable

- Manage a specific workflow (i.e. requesting access to a system, getting commitment from another team, etc)
- Planning an aspect of an event or a campaign.

Because issues can serve so many purposes and roles in GitLab, understanding where they exist and how to best utilize them is incredibly important.

Key things to know

Issues are defined in the scope of a Project. NOT a Group

1. Issues ONLY exist in projects. While Groups have boards that manage issues and lists of issues, the issues ALWAYS are stored in a project that is subordinate to the group.

1. GitLab Groups do NOT have issues. Groups have Epics.

Issue Descriptions

The top section of an issue is the body of the issue - the "Description". In the "Description", is where the core themes and topics of the issue is captured. It is not uncommon to find lengthy issue descriptions, which are used to capture the most current status of the issue.

Because the issue discussions can be long and flowing, often a best practice is to update the description with the most current/ up to date content.

Issue Details

There are many detailed parts of an issue, such as:

- Time tracking
- Weight
- Participants and notifications
- Mentions
- Related issues
- Related merge Requests
- Comments
- Zoom meetings

Issue templates

In each project, a set of "Issue Templates" can be defined to make it easy to create common issues for different situations or problems. Learn how to create an issue template

Designs and Design Discussions

Issues enable and support discussions on images and designs with Design Management. Design management makes it very useful in reviewing graphics and design work asynchronously.

Quick Actions

Issues, Merge Requests and Epics all support the ability to use keyboard shortcuts trigger key actions in the issue, MR or epic.

Quick Actions are effectively commands that make it easy and efficient to change an issue such as:

- /label followed by the label name will add a label.
- /assign followed by a user ID, will assign the issue to a person
- /close will close an issue
- /duplicate followed by an issue number will close the current issue as a duplicate and link to the given (open) issue
- /epic followed by an epic ID will assign the issue to an epic

and so on.

Confidential issues

Issues can be either open or confidential.

Default to public issues, and use the guidance on non-public communication to make a decision when in doubt.

Task lists

In the Markdown language that is used to write the description and comments in issues it is possible to create lists of items that work as checkboxes - a list of tasks

Realize that task lists are a relatively simple list of items that can be 'checked off'.

Related Issues and Blocking issues

In order to manage interdependency between issues, it is possible to create several types of relationships between issues:

1. Simple relationships. Is where one issue is simply related to another issue and there is no interdependency.
1. Blocks. Is where one issue prevents another issue from being completed.
1. Is Blocked By. is where an issue is dependent on another issue.

With issue relationships, it is possible to create a series of tasks/issues where there is a sequential requirement that one issue is completed before the following issues are completed.

Known limitations

1. An issue cannot be added to more than one milestone.

Guidelines

Use issue templates

Define and use Issue Templates for any repetitive types of issues that a team/project will need.

The issue template can be accessed directly through a URL such as <https://gitlab.com/gitlab-com/marketing/strategic-marketing/product-marketing/-/issues/new?issuabletemplate=A-SM-Support-Request>

Where the first half of the URL is the link to the project and then new?issuabletemplate= followed by the name of the template.

Use Quick Actions in the issue template to ensure that the issues have the appropriate labels, epics, and milestones associated with the issue.

Indicate a single DRI

DRI stands for Directly Responsible Individual. As such, an issue should represent a clear deliverable to be completed by the DRI. If work is required from multiple team members, with certain tasks required at different time intervals, in most cases you should break the issue into multiple issues to clarify the deliverable from each team member.

For example, if copy is needed for a landing page, and a different team member will create the page, there should be an issue for the creation of the copy and an issue for creation of the page on about.gitlab so that dependencies are clearly indicated and milestones can be applied correctly with clear DRIs for each action (copy and separately landing page creation).

Keep issue descriptions up-to-date

Issue discussions can be lengthy and exhaustive. As a best practice, update issue descriptions with the most current status and information. For example, if a discussion thread results in a better answer or improved part of the description - update the description. Don't force everyone to read the entire thread and history in order to understand the current status of the issue.

Issues maintain a history of the changes to the descriptions, so update descriptions when it makes sense.

```
{{% alert color="info" %}}
<i class="fas fa-info-circle" aria-hidden="true" style="color: rgb(49, 112, 143)
;"></i> Tip! Using a header with a clear highlight on the "latest update" can be a quick and
clear indication to any viewer what has been last agreed in conversations.
{{% /alert %}}
```

Set issue weight

Issue weight can be made relative to the tasks within an issue. Issue weight is an estimate of how much time is required to complete the tasks in the issue.

Time	Weight
-----	-----
<4 hours	0

4 hours / half a day	1	
8 hours / 1 day	2	
12 hours / 1.5 days	3	
16 hours / 2 days	4	

You can use the /weight <value> quick action to define the weight.

Use task lists sparingly

In general, avoid using markdown task lists within issues, unless they define very simple tasks, verification or approval steps. Every non-trivial task should be an issue.

Every task should be an issue

Every non-trivial task related to a Marketing program should be filed as an issue instead of a task list within the description of another issue.

When interpreted as a work unit, issues enable better planning and accountability. In particular, they can be added to milestones for scheduling purposes and to assess workload and capacity. In this regard, they also contribute to an iterative approach by having dedicated issues focused on completing a specific task only.

Use descriptive, concise issue titles that describe tasks

At a glance, everyone should be able to have a basic understanding of the task at hand by reading an issue title, and then delve into the description when more context is needed. This can contribute to scan through issue lists on GitLab.com or on e-mail efficiently.

In general, most issue titles should describe a task as a unit of work to be completed:

- · "Document existing behavior flows on about.gitlab.com" => clearly describes task and expected outcome
- · "Ultimate users" => does not provide a clear indication on the task or expectations

SECTION: marketing/project-management-guidelines/labels.md

{{< include "includes/wip-notice.md" >}}

Background

Labels are a powerful, flexible way to categorize epics, issues, and merge requests.

When applied appropriately and consistently, Labels enable GitLab users to discover, filter, manage, and report on issues, projects, or epics.

Key things to know

Label hierarchy

There are two types of labels in GitLab:

- Project labels can be assigned to issues and merge requests in that project only.
- Group labels can be assigned to epics, issues and merge requests in any project in the selected group or its subgroups.

(Learn more about Groups and Projects)

Label exclusivity

By default, Labels are not exclusive. An epic, issue, or merge request can be labeled multiple times by multiple people, supporting many different use cases and views. The exception to this rule is the Scoped Label, which specifies a set of mutually exclusive Labels. When one Scoped Label is applied, it automatically replaces any previous Label in that set. Scoped Labels are used to assign status, support workflows, or otherwise segment items into "either / or" situations.

Known limitations

Guidelines

Create Labels at the lowest possible level

Since Group-level labels cascade to every sub-group and project beneath them, unrestricted Group-level label creation can cause confusion and clutter. For example, if multiple teams create 'high priority' labels at the group level, all of them would show up in the Label menu for every project within the group.

Group-level Labels should only be used for tracking at the Group level, across multiple projects and/or subgroups. Examples of appropriate group-level labels include:

1. Executive Interest labels (e.g. CMO, CEO, etc.). This indicates the epic or issue is an executive interest item and enables reporting and visibility of related issues and epics.
1. Executive Priority (CMO-priority::1, CMO-priority::2, etc.). These labels help the organization understand the priority of issues - mainly used by CMO Staff to communicate priority on CMO's behalf.
1. Executive Theme (CMO::Agility, CMO::Efficiency, CMO::Commit, etc.). This small list of scoped labels help to flag specific CMO Interest themes based on top level initiatives and areas of focus
1. "Team" Labels. (Product and Solution Marketing, Corporate Marketing, etc.). This allows high-level work tracking within the greater Marketing organization.

If you are unsure whether you should create a group-level label or a project-level Label, err on the side of caution and create a project-level Label.

You can always promote it to a group-level Label in the future.

Limit Label name character count

When creating a Label name, use the smallest possible number of characters. Labels longer than 25 characters may be truncated in the GitLab UI.

Provide detailed descriptions

When you create a Label, ensure that the Description field contains the following information:

- When the Label should be applied
- Purpose or Goal
- DRI

This information will be presented in a tooltip above the Label throughout the UI.

Group like Labels with naming conventions

If you are creating a collection of related Labels, group them using common prefix in the name. For example, if you are creating 'West' and 'East' to describe territories, consider Labels such as 'SalesRegionWest' and 'SalesRegionEast' instead. This will provide immediate context to observers while also grouping your Labels together in the UI.

Templatize Label creation

Labels only work properly when they are applied consistently. Wherever possible, automate the creation of appropriate Labels by adding them to Issue templates.

Apply Scoped Labels to remove confusion

To minimize clutter and reduce the possibility of incorrect double-counts, always consider creating Scoped Labels to automate Label management, rather than depending on users to remove invalid labels as they assign new ones.

Automate Label Hygiene

The Triage Bot is an open source project that makes it possible to automate many issue and merge request hygiene tasks to ensure that your projects are more consistently following a process. Use the Triage Bot scripts to minimize the impact of human error, automating proper Labeling and workflow. Common uses of the Triage Bot include identifying missing or improperly applied Labels, notifying issue or merge request owners of Label-related problems, and applying Labels based on predefined criteria.

For more information, please refer to [@johnjeremiah's YouTube tutorial](#).

Don't Delete, Deprecate

Deleting a Label will remove all prior associations of that Label with issues or merge requests, which can, in turn, break dependencies. All Label deletions should be performed by the Label's DRI (should be listed in the Label description, as stated above).

If you are the DRI and wish to delete a Label, follow the following steps:

- Append the 'DEPRECATE' tag to your Label (e.g., 'pMm' becomes 'DEPRECATEpMm').
- Socialize the change within your group.
- Wait a calendar month before deleting the Label.

SECTION: marketing/project-management-guidelines/milestones.md

{{< include "includes/wip-notice.md" >}}

Background

There are two concepts of time-based tracking in GitLab.

Milestones: useful for tracking the progress of multiple issues across a specific time period, and when planning and managing epics.

Iterations: useful for planning agile or agile-like sprints to capture action items to be completed within a specific time period.

Milestones

Milestones are a great way to track the progress of multiple related issues across a specific time period. With milestones, you can see how fast issues are being completed in that time period (burndown chart), and you can view the issues grouped by labels, and grouped by status (unassigned, assigned, and completed)

Milestones are very useful when tracking the progress of multiple issues and when planning and managing epics.

Here are two examples of milestones:

1. Product and Solution Marketing Quarterly Milestone - showing the bulk of strategic marketing work for a given quarter.
1. UseCase Monthly Sprint Milestone

Key things to know

1. Milestones can be created at both Project and Group levels.
1. Milestones can be used to establish dates for epics (where the epic inherits the dates from

the issues and associated milestones)

1. Milestones set a date range (start date and end date)

1. Issues are added to milestones

1. Milestone lists can be sorted only by name and date (either due date or start date)

Known limitations

1. Milestone lists cannot be filtered and have limited sorting

1. Milestones do not have labels and cannot be filtered by labels

1. Milestones do not track history of changes or who created/changed the milestone

1. On issues - when adding an issue to a milestone, the list of milestones is ALL of the milestones in that project and the parent groups (potentially LOTS AND LOTS of milestones)

1. On groups - the list of milestones is ALL of the milestones defined in that group and below that group (potentially LOTS AND LOTS of milestones) - This feature request addresses this limitation.

1. On Projects - the list of milestones is ONLY the milestones defined in that project (deceptively very